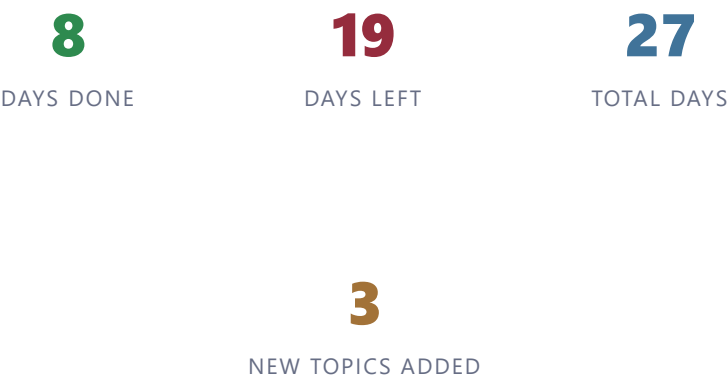


ACCENTURE AEH · INTERNSHIP FY26

Python Training Plan

Revised & Remaining

Updated plan for the remaining internship days — adjusted for completed progress, new mandatory topics, and final review deadline.



- Git & GitHub
- MySQL
- PostgreSQL
- SQLAlchemy
- FastAPI
- JWT Auth
- Redis
- Celery
- ★ GenAI
- ★ Agentic AI
- ★ App Security

Project Start: Jun 4, 2026 · Today: Jun 15, 2026 · Last Day: Jul 10, 2026 · Final Review Deadline: Jul 10, 2026

Internship Progress

Jun 4 — Start

Jun 15 — Today

Jul 10 — End

8 days completed (30%) | 19 days remaining (70%)

Completed — Days 1–8

Day	Date	Topic	Status
Day 1	Thu Jun 04	Python Basics Refresh — Variables, Strings, Lists, Dicts, Loops, Functions	✓ Done
Day 2	Fri Jun 05	Collections Module + Comprehensions & Generators	✓ Done
Day 3	Mon Jun 08	OOP Part 1 — Classes, __init__, __str__, @property	✓ Done
Day 4	Tue Jun 09	OOP Part 2 — Inheritance, Polymorphism + Functional Tools (map, filter, lambda)	✓ Done
Day 5	Wed Jun 10	Mini Project — Contact Book (OOP + Collections + Functional)	✓ Done
Day 6	Thu Jun 11	File Handling + Exception Handling (CSV, custom exceptions)	✓ Done
Day 7	Fri Jun 12	JSON Handling + Logging (nested JSON, RotatingFileHandler)	✓ Done
Day 8	Mon Jun 15	Regular Expressions (validate email, phone, extract dates, mask card numbers)	✓ Done

★ 3 New Topics Added (from Accenture program requirements)

- NEW · Day 9 **Git & GitHub** — Critical for all project work. Every intern needs this from Day 1 of project deployment.
- NEW · Day 24 **Intro to GenAI with Python** — Accenture Week 1 mandatory topic. Calling LLM APIs, prompt engineering, building simple GenAI features.

NEW · Day 25

Agentic AI with Python — Accenture Week 1 mandatory topic. LangChain basics, building a simple AI agent with tools. Application Security basics also woven in.

Remaining Plan — Days 9–27 (Jun 16 → Jul 10)

Day	Date	Topic
Day 9	Tue Jun 16	★ NEW — Git & GitHub Basics
Day 10	Wed Jun 17	Modules, Packages & Virtual Environments
Day 11	Thu Jun 18	Decorators & Closures + Itertools & Functools
Day 12	Fri Jun 19	SMTP — Sending Emails
Day 13	Mon Jun 22	Working with APIs + Application Security Basics
Day 14	Tue Jun 23	Multithreading + Testing with pytest
Day 15	Wed Jun 24	Mini Project — Sales Report Automation
Day 16	Thu Jun 25	MySQL with Python
Day 17	Fri Jun 26	PostgreSQL with Python
Day 18	Mon Jun 29	SQLAlchemy ORM
Day 19	Tue Jun 30	Pandas + Data Visualization
Day 20	Wed Jul 01	FastAPI Part 1 + Environment Variables
Day 21	Thu Jul 02	FastAPI Part 2 + JWT Authentication
Day 22	Fri Jul 03	Redis + Caching
Day 23	Mon Jul 06	Celery + Cron Jobs
Day 24	Tue Jul 07	★ NEW — Intro to GenAI with Python
Day 25	Wed Jul 08	★ NEW — Agentic AI with Python
Day 26	Thu Jul 09	Capstone Project — HR Management System PoC
Day 27	Fri Jul 10	Final Review Preparation + PPO Presentation

REMAINING WEEK 2 / 5

Core Tools + Git

June 16 – June 19, 2026

Git is critical before any project work. Then round off core Python tools with modules, decorators, and SMTP.

Day 9 — Git & GitHub ★

Day 10 — Modules & venv

Day 11 — Decorators + Itertools

Day 12 — SMTP

DAY 09 • NEW**Git & GitHub Basics**

Tue Jun 16

TOPICS

- › git init, clone, status, add, commit, push, pull
- › Branching: branch, checkout, merge
- › Pull Requests and code review flow
- › .gitignore — what to never commit (.env, __pycache__)
- › Resolving merge conflicts
- › Git workflow used in Accenture projects (feature branches)

DAILY TASK

- ✓ Create a GitHub repo for the internship project
- ✓ Push all code written so far (Days 1-8) to the repo
- ✓ Create a dev branch, make a change, raise a PR to main
- ✓ Add a proper .gitignore (exclude .env, __pycache__, *.pyc, venv/)
- ✓ Write a README.md with project description and setup steps

```
git init git add . git commit -m  
"initial commit" git checkout -b  
feature/day9-git git push origin  
feature/day9-git # Open PR on GitHub  
→ merge to main
```

DAY 10 Modules, Packages & Virtual Environments

Wed Jun 17

TOPICS

- › Creating and importing custom modules
- › __name__ == "__main__" pattern
- › venv: create, activate, deactivate
- › pip: install, freeze, requirements.txt
- › os, sys, datetime, random standard library

DAILY TASK

- ✓ Create utils.py with helper functions (age from DOB, random password generator, file size checker)
- ✓ Import and use all in main.py
- ✓ Set up a clean virtual environment for the project
- ✓ Generate requirements.txt with pip freeze
- ✓ Commit utils.py and requirements.txt to GitHub (Day 9 repo)

DAY 11 **Decorators & Closures + Itertools & Functools**

Thu Jun 18

TOPICS

- › Closures — function remembering outer scope
- › Writing decorators with @, chaining, functools.wraps
- › itertools: combinations, groupby, islice, chain
- › functools: reduce, partial, lru_cache

DAILY TASK

- ✓ Write @timer (execution time) and @logger (fn name + args) decorators
- ✓ Use lru_cache on fibonacci — compare time with @timer
- ✓ Use combinations to generate team fixtures from 5 teams
- ✓ Use groupby to group transactions by category
- ✓ Push to GitHub with a descriptive commit message

DAY 12 **SMTP — Sending Emails**

Fri Jun 19

TOPICS

- › smtplib: SMTP_SSL, plain text emails
- › HTML emails with MIMEMultipart + MIMEText
- › File attachments with MIMEBase + encoders
- › Gmail SMTP with App Password
- › Store credentials in .env — never hardcode
- › Sending to multiple recipients

DAILY TASK

- ✓ Send plain text email via Gmail SMTP
- ✓ Send an HTML styled report email
- ✓ Attach report.csv (from Day 6) as file attachment
- ✓ Store SMTP credentials in .env using python-dotenv
- ✓ Log sent/failed status per recipient

WEEK 3 / 5

Advanced Python + Databases Begin

June 22 – June 26, 2026

APIs, security, threading, testing — then kick off databases with MySQL and PostgreSQL.

Day 13 — APIs + Security

Day 14 — Threads + pytest

Day 15 — Mini Project

Day 16 — MySQL

Day 17 — PostgreSQL

DAY 13 Working with APIs + Application Security Basics

Mon Jun 22

TOPICS

- › requests: GET, POST, PUT, DELETE, headers, timeout
- › Parsing JSON responses, HTTP status codes
- › Session objects for multiple requests
- › **Security:** Never expose API keys in code
- › **Security:** Input validation — never trust user input
- › **Security:** HTTPS only, check SSL certs
- › **Security:** OWASP Top 5 basics for developers

DAILY TASK

- ✓ Call `jsonplaceholder.typicode.com/users` — GET, POST, PUT, DELETE
- ✓ Save results to `api_results.json`
- ✓ Store all API keys and URLs in `.env` (not in code)
- ✓ Add input validation before making API calls
- ✓ Log all requests with timestamps

DAY 14 Multithreading + Testing with pytest

Tue Jun 23

TOPICS

- › `ThreadPoolExecutor` for concurrent tasks
- › `threading.Lock` for shared resource safety
- › GIL concept — threads vs processes
- › `pytest`: fixtures, parametrize, assertions
- › `unittest.mock` for mocking external calls
- › Running tests: `pytest -v`

DAILY TASK

- ✓ Fetch 5 URLs sequentially — measure time
- ✓ Repeat with `ThreadPoolExecutor` — compare time
- ✓ Use `Lock` to safely write results to shared log
- ✓ Write `pytest` suite for `BankAccount` (Day 3) with `parametrize`
- ✓ Mock an external API call and test the dependent function

DAY 15 Mini Project — Daily Sales Report Automation

Wed Jun 24

MINI PROJECT

- ✓ Read sales data from a JSON file
- ✓ Calculate: total sales, top 5 products, department summary
- ✓ Generate an HTML formatted report
- ✓ Send HTML email + CSV attachment via SMTP to multiple recipients
- ✓ Use `ThreadPoolExecutor` to send emails concurrently
- ✓ Decorate key functions with `@timer` and `@logger`
- ✓ Log everything to `automation.log`
- ✓ Push complete project to GitHub with a proper PR

DAY 16 MySQL with Python

Thu Jun 25

Real-World Use: E-commerce, CMS, user management, transactional data systems.**TOPICS**

- › mysql-connector-python / pymysql
- › Connect, create DB and tables
- › CRUD: INSERT, SELECT, UPDATE, DELETE
- › Parameterized queries (prevent SQL injection)
- › .env for DB credentials

DAILY TASK

- ✓ Connect Python to MySQL — create employees table
- ✓ Write: add employee, get by dept, update salary, delete by ID
- ✓ Use ONLY parameterized queries
- ✓ Store credentials in .env
- ✓ Log all DB operations to db.log

DAY 17 PostgreSQL with Python

Fri Jun 26

Real-World Use: Fintech, enterprise backends, analytics platforms — preferred for complex queries and ACID compliance.**TOPICS**

- › psycopg2: connect, cursor, execute
- › CRUD operations
- › Transactions: commit(), rollback()
- › PostgreSQL-specific types: SERIAL, JSONB, TIMESTAMP
- › Key differences vs MySQL

DAILY TASK

- ✓ Connect to PostgreSQL — create orders table (SERIAL, TIMESTAMP)
- ✓ Write: place order, get by customer, update status, delete
- ✓ Use transactions — rollback on any failure
- ✓ Store JSONB metadata field per order
- ✓ Push to GitHub — compare MySQL vs PostgreSQL code in README

WEEK 4 / 5

ORM, Pandas & FastAPI

June 29 – July 3, 2026

SQLAlchemy ORM, data analysis, and build the full FastAPI backend with authentication and caching.

Day 18 — SQLAlchemy ORM

Day 19 — Pandas + Viz

Day 20 — FastAPI Part 1

Day 21 — FastAPI + JWT

Day 22 — Redis

DAY 18 SQLAlchemy ORM

Mon Jun 29

Real-World Use: Industry standard ORM for Python backends — used in Flask-SQLAlchemy, FastAPI projects. Raw SQL is rarely written directly in production.

TOPICS

- › What is ORM and why industry uses it
- › Engine, Session, declarative_base
- › Defining models as Python classes
- › CRUD via ORM (no raw SQL)
- › ForeignKey and relationship() between models
- › Switch DB URL between MySQL and PostgreSQL

DAILY TASK

- ✓ Define Department and Employee ORM models with ForeignKey
- ✓ Write ORM functions: add, list, update, delete
- ✓ Query all employees in a department using relationship()
- ✓ Switch DB URL from MySQL to PostgreSQL — verify no code changes
- ✓ Push to GitHub

```
class Employee(Base): __tablename__ =
    "employees" id = Column(Integer,
        primary_key=True) name =
        Column(String(100)) dept_id =
        Column(Integer,
            ForeignKey("departments.id"))
        department =
            relationship("Department")
```

DAY 19 Pandas + Data Visualization

Tue Jun 30

Real-World Use: ETL pipelines, HR analytics, sales dashboards, report generation, data cleaning before DB inserts.

TOPICS

- › DataFrame CRUD, read_csv, read_json, to_csv
- › groupby, merge, pivot_table, apply
- › Handling nulls: fillna, dropna
- › matplotlib + seaborn: bar, histogram, scatter, heatmap
- › Saving charts as PNG

DAILY TASK

- ✓ Load employee data from PostgreSQL via SQLAlchemy into Pandas DataFrame
- ✓ Find: average salary by dept, top 5 earners
- ✓ Fill missing salary values with dept median
- ✓ Generate 2×2 dashboard: salary distribution, headcount by dept, salary vs experience scatter, correlation heatmap
- ✓ Save as hr_dashboard.png — push to GitHub

DAY 20 FastAPI Part 1 + Environment Variables

Wed Jul 01

Real-World Use: Most adopted Python API framework in industry — microservices, SaaS backends, internal tools.

TOPICS

- › python-dotenv: .env, load_dotenv(), os.getenv()
- › Why .env is mandatory in production
- › FastAPI: GET, POST, path & query params
- › Request body with pydantic BaseModel
- › Swagger UI at /docs (auto-generated)
- › Connect FastAPI to PostgreSQL via SQLAlchemy

DAILY TASK

- ✓ GET /employees — list all
- ✓ GET /employees/{id} — get one
- ✓ POST /employees — add new
- ✓ All credentials from .env
- ✓ Test on <http://127.0.0.1:8000/docs> (Swagger)
- ✓ Log every request — push to GitHub

DAY 21 FastAPI Part 2 + JWT Authentication

Thu Jul 02

Real-World Use: JWT is the industry standard for securing REST APIs — used in every microservice, mobile backend, and SaaS platform.

TOPICS

- › PUT and DELETE routes
- › JWT — how it works (header.payload.signature)
- › python-jose for encoding/decoding tokens
- › passlib[bcrypt] for password hashing
- › Login route returning access token
- › Protected routes with Depends() + token verification

DAILY TASK

- ✓ PUT /employees/{id} — update (JWT protected)
- ✓ DELETE /employees/{id} — delete (JWT protected)
- ✓ POST /auth/register — hash password with bcrypt
- ✓ POST /auth/login — returns JWT access token
- ✓ Return 401 for invalid/missing tokens

DAY 22 Redis + Caching

Fri Jul 03

Real-World Use: Redis is in virtually every large-scale production system. Caching reduces DB load by 80–90% on read-heavy APIs.

TOPICS

- › What is Redis and why caching matters
- › redis-py: set, get, delete, expire (TTL)
- › Caching API responses to reduce DB load
- › Cache invalidation on data change
- › Storing JWT sessions in Redis

DAILY TASK

- ✓ Cache GET /employees response for 60 seconds
- ✓ Cache GET /employees/{id} with per-record TTL
- ✓ Invalidate cache on every PUT/DELETE
- ✓ Store JWT session in Redis after login
- ✓ Log "CACHE HIT" vs "CACHE MISS" per request

WEEK 5 / 5 — FINAL WEEK

GenAI, Celery & Final Review

July 6 – July 10, 2026

Celery for background jobs, GenAI and Agentic AI with Python, Capstone PoC, and PPO final review preparation.

Day 23 — Celery

Day 24 — GenAI ★

Day 25 — Agentic AI ★

Day 26 — Capstone PoC

Day 27 — Final Review Prep

DAY 23 Celery + Cron Jobs

Mon Jul 06

Real-World Use: Celery is the industry standard for background tasks — async emails, report generation, file processing, nightly DB exports. Used in every production Django/FastAPI app.

TOPICS

- › What is Celery (distributed task queue)
- › Celery with Redis as broker
- › `@app.task` — async background tasks
- › `.delay()` and `.apply_async()` to trigger tasks
- › celery beat — periodic/scheduled tasks
- › Flower dashboard for task monitoring

DAILY TASK

- ✓ Async task: send welcome email when new employee added via POST
- ✓ Scheduled task (celery beat): every 2 mins generate dept summary → log
- ✓ Daily midnight task: export all employees to JSON backup
- ✓ Monitor tasks on Flower at <http://localhost:5555>

DAY 24 • NEW Intro to GenAI with Python

Tue Jul 07

Accenture Mandatory: Intro to GenAI is part of Week 1 centralized learning. This day applies it practically in Python for project use cases.

TOPICS

- › What is GenAI — LLMs, tokens, prompts, completions
- › Calling LLM APIs from Python (OpenAI / Gemini API)
- › Prompt engineering basics: system, user, assistant roles
- › Structured outputs — asking LLM to return JSON
- › Streaming responses
- › API keys in `.env` — never in code

DAILY TASK

- ✓ Call an LLM API to generate a job description from a role title
- ✓ Ask LLM to return structured JSON (employee performance summary)
- ✓ Add a FastAPI route: POST `/ai/summarize` — accepts employee data, returns AI-generated summary
- ✓ Store API key in `.env`
- ✓ Log all API calls and token usage

```
from openai import OpenAI client =
OpenAI(api_key=os.getenv("OPENAI_API_KEY"))
response = client.chat.completions.create(
model="gpt-4o-mini", messages=
[{"role": "user", "content": prompt}] )
```

DAY 25 • NEW **Agentic AI with Python**

Wed Jul 08

Accenture Mandatory: Intro to Agentic AI is a Week 1 mandatory topic. Accenture is actively building GenAI accelerators — this is directly relevant to project work.

TOPICS

- › What is an AI Agent — perception, decision, action loop
- › LangChain basics: LLM, prompt templates, chains
- › Tools — giving the agent abilities (search, calculate, query DB)
- › ReAct pattern (Reasoning + Acting)
- › Simple agent that uses tools to answer questions
- › Difference: LLM call vs Agent vs RAG

DAILY TASK

- ✓ Build a simple HR Assistant agent using LangChain
- ✓ Give it 2 tools: `get_employee_count(dept)` and `get_avg_salary(dept)` — both query the PostgreSQL DB
- ✓ Agent answers questions like "How many engineers do we have and what is their average salary?"
- ✓ Add a FastAPI route: `POST /ai/ask` — accepts a question, returns agent's answer
- ✓ Test with 5 different questions — push final project to GitHub

```
from langchain.agents import
initialize_agent, Tool from
langchain.llms import OpenAI tools =
[Tool(name="GetEmployeeCount",
func=get_count, ...)] agent =
initialize_agent(tools, llm,
agent="zero-shot-react-description")
result = agent.run("How many people
in Engineering?")
```


DAY 26 Capstone Project — HR Management System PoC

Thu Jul 09

HR Management System — Production PoC ————— PostgreSQL
 + SQLAlchemy ORM — departments, employees, users, audit_logs FastAPI REST API
 — POST /auth/register & /auth/login (JWT) — GET /employees (Redis cached, 60s
 TTL) — POST /employees (triggers Celery welcome email) — PUT /employees/{id}
 (JWT protected) — DELETE /employees/{id} (JWT protected) — POST /ai/summarize
 (GenAI — employee summary) — POST /ai/ask (Agentic AI — HR assistant) Redis →
 Cache all GETs + JWT sessions Celery → Async email + daily JSON backup Pandas →
 Salary analytics from DB Matplotlib → HR dashboard PNG saved to disk SMTP → Report
 email with chart attachment Logging → All events to rotating app.log .env → ALL
 credentials (never hardcoded) Git → Full history on GitHub with PRs

Database

PostgreSQL + SQLAlchemy ORM. Relationships between departments and employees. No raw SQL.

API

FastAPI with full CRUD. JWT protected. Pydantic validation. Auto Swagger docs.

Performance

Redis caches all reads. Cache invalidated on writes. JWT sessions in Redis.

Background Jobs

Celery async email on hire. Celery beat for scheduled reports and backups.

GenAI Feature

AI-generated employee summaries. HR assistant agent queries the live DB.

Quality

Full logging. All secrets in .env. GitHub repo with clean commit history.

DAY 27 Final Review Preparation + PPO Presentation

Fri Jul 10 — LAST DAY

The final review is conducted by Empaneled Interviewers (CL 8+) on HirePro. PPO is determined from this review. Prepare to present the Capstone PoC and demonstrate the 6 evaluation parameters.

HirePro Evaluation Parameters (scored 1–5)

- Intellectual Ability — problem solving, analytical skills
- Leadership Qualities — confidence, decisiveness
- Team Working — flexibility, networking
- Initiative — self-starter, ownership
- Interpersonal Attributes — communication, receptive to feedback
- Continuous Learner & Agile — curious, embraces change

Final Review Preparation Tasks

- Prepare a 5-slide presentation: Problem → Solution → Architecture → Demo → Learnings
- Live demo of the FastAPI + GenAI PoC
- Explain every technical decision made (why PostgreSQL, why Redis, why JWT)
- Share GitHub repo link — clean commit history shows work progress
- Prepare 3 challenges faced and how they were solved
- Practice answers for "What would you do differently?"

Parameter	How to Score High
Intellectual Ability	Explain the why behind architecture choices — not just what you built
Leadership	Show you took ownership of the PoC end-to-end without being told what to do
Team Working	Mention collaboration with buddy, standups, code reviews on GitHub PRs
Initiative	Highlight the 3 new topics added (Git, GenAI, Agentic AI) beyond the original plan
Interpersonal	Clear, structured presentation — listen carefully to interviewer questions
Continuous Learner	Show daily GitHub commits — consistent learning over 27 days

Environment Setup

Run once at the start. All libraries needed for remaining Days 9–27.

```
# Core libraries pip install mysql-connector-python psycpg2-binary sqlalchemy \ fastapi
uvicorn python-jose passlib[bcrypt] \ redis celery flower python-dotenv \ pandas
matplotlib seaborn requests pytest pydantic # GenAI / Agentic AI (Day 24-25) pip install
openai langchain langchain-openai # .env template DB_HOST=localhost DB_USER=root
DB_PASSWORD=yourpassword PG_HOST=localhost PG_DB=internship_db PG_USER=postgres
PG_PASSWORD=yourpassword REDIS_HOST=localhost REDIS_URL=redis://localhost:6379/0
SECRET_KEY=your-jwt-secret-key SMTP_EMAIL=youremail@gmail.com SMTP_PASSWORD=your-app-
password OPENAI_API_KEY=sk-... # Run commands uvicorn main:app --reload # FastAPI celery
-A tasks worker --loglevel=info # Celery worker celery -A tasks beat --loglevel=info #
Celery scheduler celery -A tasks flower --port=5555 # Task monitor
```

Remaining Timeline at a Glance

Week	Dates	Focus	Days
✓ Done	Jun 4–15	Python Foundations, OOP, Files, JSON, Logging, Regex	1–8
Week 2 (rem)	Jun 16–19	Git, Modules, Decorators, SMTP	9–12
Week 3	Jun 22–26	APIs + Security, Threads, pytest, MySQL, PostgreSQL	13–17
Week 4	Jun 29–Jul 3	SQLAlchemy, Pandas, FastAPI, JWT, Redis	18–22
Week 5 Final	Jul 6–10	Celery, GenAI, Agentic AI, Capstone PoC, PPO Review	23–27